

Plan de développement

Etudiant : Jérémy, Ahmed, Jonathan & Antoine - (300-500 mots / Max 4 Tâches)

Introduction

Ce projet vise à concevoir une plateforme de détection d'anomalies zero-day dans un environnement Kubernetes, en exploitant l'observabilité bas niveau offerte par eBPF. L'objectif scientifique est d'évaluer si la modélisation des comportements système sous forme de graphes dynamiques permet d'identifier des activités malveillantes inédites sans signature préalable. L'architecture repose sur la collecte d'événements noyau via Tetragon, leur transformation en structures exploitables, puis leur analyse algorithmique pour détecter des déviations comportementales. Le développement est structuré en quatre tâches progressives couvrant cadrage, implémentation, validation et consolidation.

Tâche 1 : Cadrage scientifique et architectural

Cette première phase établit les bases conceptuelles et techniques. Elle comprend une étude approfondie d'eBPF et du fonctionnement de Tetragon en environnement Kubernetes (déploiement DaemonSet, TracingPolicies). Un état de l'art sur la détection d'anomalies basée sur graphes est réalisé, ainsi qu'une analyse des scénarios d'attaque inspirés du framework MITRE ATT&CK.

Les livrables incluent un cahier des charges détaillé, un schéma d'architecture globale (collecte, traitement, stockage, analyse) et un planning précis. Cette phase garantit la cohérence scientifique et la faisabilité technique du projet.

Tâche 2 : Implémentation de l'infrastructure et du pipeline

Cette phase correspond au cœur du développement. L'infrastructure Kubernetes est déployée localement afin d'héberger Tetragon et les composants applicatifs. Les événements système collectés sont transformés en graphes dynamiques à l'aide de NetworkX.

Un pipeline Python assure l'ingestion, la normalisation et la mise à jour continue des graphes comportementaux. Un modèle de référence (baseline) représentant le comportement normal est construit. Des métriques d'évaluation sont définies : précision, rappel et taux de faux positifs. Cette étape permet d'obtenir un prototype fonctionnel.

Tâche 3 : Validation expérimentale et tests offensifs

La troisième phase consiste à évaluer la robustesse du système. Des scénarios d'attaque contrôlés (reverse shell, élévation de privilèges, exfiltration) sont exécutés afin d'observer la capacité du modèle à détecter des déviations.

Une analyse quantitative est menée à partir des métriques définies précédemment. Les cas de non-détection sont étudiés afin d'identifier les limites structurelles du modèle et d'améliorer les critères de comparaison de graphes.

Tâche 4 : Optimisation et finalisation

La dernière phase vise l'optimisation des performances (complexité des graphes, latence d'analyse) et l'amélioration de la visualisation via un tableau de bord interactif. Le code est documenté pour garantir la reproductibilité. Le rapport final consolide l'ensemble des résultats expérimentaux et des choix méthodologiques.